Optimizing Multi-Hop Reasoning in E-Business Information Systems through Agentic Graph RAG with Efficient Reranking*

Mahfuja Yesmin[1], Sabbir Ahmed[2], Mohammad Shahadat Hossain[1], and Karl Andersson[3]

[1] Department of Computer Science and Engineering, University of Chittagong, Bangladesh
[2] ECSS, University of Texas at Dallas, USA
[3] Department of Computer Science, Electrical and Space Engineering, Lulea University of Technology, Skelleftea, Sweden
{mahfujasolaiman77, sabbir.iit.ju}@gmail.com

Abstract

Traditional Retrieval Augmented Generation (RAG) systems often struggle with multi-hop reasoning and efficiency, limiting their ability to deliver accurate and context-rich responses in E-Business Information Systems. Although Graph RAG improves retrieval by leveraging structured knowledge, it still faces challenges in relevance, scalability, and latency. We propose an Agentic Graph RAG framework, where reasoning-model-based agents (o4-mini, Gemini-2.5-Flash) decompose queries and prune irrelevant paths, and rerankers (bge-reranker-base, ms-marco-MiniLM-L6-v2) enhance the quality of retrieved contexts. By addressing noise, computational overhead, and irrelevant retrieval, our framework effectively overcomes the limitations of traditional RAG and Graph RAG for knowledge-intensive digital business activities. We assess our method using three benchmarks: LegalBench-RAG-mini, which represents high-stakes analytical reasoning; HotpotQA, which evaluates generalizability across broader knowledge domains; and FinanceBench, a publicly accessible financial QA dataset based on real-world corporate disclosures and earnings reports that directly reflects the information demands of E-Business environments. Results demonstrate that our framework consistently outperforms Graph RAG baselines in retrieval efficacy and answer correctness while maintaining practical efficiency, demonstrating its potential as an adaptable and interpretable solution for multi-hop reasoning in domains where accuracy and reliable decision-making are critical.

1 Introduction

Rapid development of Large Language Models (LLMs) has transformed natural language processing, allowing human-like text generation [32]. Traditional LLMs, on the other hand, are prone to hallucinations, outdated information, and domain constraints, making them unsuitable for knowledge-intensive activities in E-Business Information Systems and other business applications. Retrieval Augmented Generation (RAG) tackles these challenges by incorporating external knowledge retrieval, thus improving factual consistency and response legitimacy [6]. However, deeper semantic relationships are frequently missed by conventional RAG architectures, which rely on vector-based dense retrieval. Consequently, they encounter significant

*Proceedings of The 2026 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution (EBISION 2026), Article No. 32, July 8–10, 2026, Seoul, South Korea.
© The copyright of this paper remains with the author(s).

efficiency bottlenecks when scaled to huge knowledge bases, suffer from noisy or partially relevant retrievals, and struggle with multi-hop reasoning [24]. These limitations are particularly evident in financial information systems, a critical domain within E-Business, where corporate filings, earnings reports, and regulatory disclosures are lengthy, complex documents that include narrative prose, numerical tables, and domain-specific terms, making simple retrieval far too inadequate for the multi-step queries that business professionals frequently encounter [8]. Resolving questions about year-over-year revenue fluctuations or cross-segment spending breakdowns necessitates integrating diverse data across dozens of document sections, a task that requires systematic, multi-hop reasoning rather than a single lookup [22]. The sheer volume and heterogeneity of such data make it difficult to analyze automatically and costly to interpret manually, but clarity and precision are essential in corporate contexts where procurement, compliance, and investment choices are dependent on reliable data access [3].

These difficulties underline the necessity of retrieval frameworks that can link scattered data across documents while preserving precision and effectiveness in complex reasoning tasks. To overcome some of these constraints, Graph RAG structures the retrieval procedure around knowledge graphs, which allow for multi-hop reasoning and the explicit modeling of entity relationships [18]. This approach does not completely address the efficiency–accuracy trade-offs, since Graph RAG still has issues with scalability constraints, noisy or incomplete graph construction, and high latency during graph traversal and reasoning [26].

Agentic RAG addresses this issue by introducing autonomous agents that enhance retrieval, iteratively analyze contextual information, and optimize decision-making [21]. When combined with reasoning models, this method enhances response accuracy by eliminating inconsistencies and boosting contextual coherence, resulting in more trustworthy decision-making in complex queries. Integrating the agentic approach with Graph RAG (i.e. Agentic Graph RAG) shows how incorporating agentic strategies into graph-based retrieval allows for more efficient multi-hop reasoning and dynamic exploration of relational knowledge, combining the benefits of structured knowledge with more accurate and adaptive decision-making [14].

To address these shortcomings, we introduce an Agentic Graph RAG framework that combines effective rerankers with lightweight reasoning model-based agents. While rerankers polish retrieved contexts to eliminate noise and redundancy, agents guide multi-hop query decomposition and prune irrelevant paths. This multi-layered method balances retrieval quality, efficiency, and interpretability while significantly lowering reranking overhead and improving retrieval precision for E-Business Information Systems and other knowledge-intensive domains. Our framework is designed to answer the following two research questions:

- **RQ1:** How can reranking bottlenecks in the Agentic Graph RAG be optimized, and what are the efficiency-performance trade-offs when incorporating reasoning models?
- **RQ2:** How can different combinations of reasoning models and reranking techniques in hybrid Agentic Graph RAG systems impact retrieval quality, decision-making, and efficiency for multi-hop queries under practical efficiency constraints?

These research questions seek to improve Agentic Graph RAG systems by resolving reranking bottlenecks and exploring reasoning-reranker synergies, resulting in more accurate and practically efficient multi-hop retrieval — a quality critical in business domains dealing with complex and sensitive data, where response quality directly influences decision-making, risk management, and system reliability. Our evaluations on FinanceBench, LegalBench-RAG-mini, and HotpotQA show that the framework extends far beyond any single domain, making it especially relevant to E-Business Information Systems, where accurate, timely, and trustworthy multi-hop knowledge access has a direct impact on operational and strategic decision-making.

2 Related Work

Graph RAG. RAG research has taken a variety of directions, including graph-based retrieval, agentic retrieval, hybrid agentic-graph systems, and reranking techniques. Traditional RAG pipelines that use vector-based retrieval frequently struggle with noisy environments and multi-hop reasoning. To address these limitations, graph-based approaches such as KG^{2}RAG [34], PathRAG [2], MiniRAG [5], and NodeRAG [27] leverage knowledge graphs to capture entity relationships and enhance reasoning. These methods improve factual accuracy and interpretability, but have significant graph construction expenses, traversal latency, and are susceptible to noise in graph production. While Graph RAG demonstrates stronger reasoning capabilities, it struggles to reconcile efficiency and scalability, particularly when used with large or complicated corpora [18]. This further constrains its use in data-intensive applications like E-Business systems, where precise and rapid information retrieval is crucial.

Agentic RAG and Agentic Graph RAG. While Graph RAG focuses on structured knowledge representation, Agentic RAG broadens the paradigm by incorporating autonomous reasoning and decision-making capabilities. Agentic RAG utilizes LLM-driven agents to dynamically decompose questions, filter noise, and coordinate reasoning. Systems such as MAIN-RAG [1], AU-RAG [9], and MA-RAG [15] apply multi-agent filtering, domain-generalized retrieval, and collaborative reasoning to improve adaptability and accuracy. However, these methods often add processing overhead and risk propagating hallucinations, which are unacceptable in finance-related domains. Graph-R1 [14] introduces reinforcement learning to optimize graph-based retrieval and reasoning in an end-to-end manner, allowing the model to learn effective retrieval strategies through iterative feedback. By investigating relational paths and producing intermediate reasoning steps, Think-on-Graph [23] allows LLMs to carry out explicit reasoning over graph structures. To facilitate adaptive retrieval and reasoning in changing information contexts, AGENTiGraph [33] combines autonomous agents with dynamic knowledge graphs. Despite their strong reasoning abilities, these systems frequently need sophisticated graph management techniques, large computational resources, or intricate optimization processes.

Reranking Strategy. Beyond improvements in retrieval and reasoning, reranking has emerged as an important step toward maintaining relevance. Early work such as Re^{2}G [7] introduced the retrieve–rerank–generate paradigm, showing that intermediate reranking improves response quality. More current approaches use varied reranking strategies: G-RAG [4] combines graph-based context refinement, RankRAG [30] unifies ranking and generation, JudgeRank [16] uses LLMs for reasoning-intensive reranking, and REARANK [31] uses reinforcement learning to optimize reranking decisions. However, these approaches often incur high latency and efficiency costs, limiting their applicability in time-sensitive systems.

Overall, prior work improves reasoning and retrieval relevance but fails to fully address the trade-offs between accuracy, efficiency, and scalability. These limitations are particularly critical in E-Business Information Systems, which require timely and reliable information retrieval. These shortcomings motivate our proposed Agentic Graph RAG framework, which integrates lightweight reasoning-model-based agents and efficient neural rerankers within a structured multi-stage pipeline comprising query decomposition, path pruning, and reranking. This enables iterative refinement that reduces noise and computational overhead while achieving robust multi-hop reasoning and strong retrieval performance under practical efficiency constraints.

3 Methodology

In this section, we examine the preliminary step of generating knowledge-based graphs from our text-based corpus, followed by a detailed explanation of the framework's working principles and components for efficient multi-hop reasoning.

3.1 Preliminaries

The starting point in our approach is to create an indexing graph from the text corpus. Inspired by PathRAG [2], we use an LLM-based graph generation method to convert unstructured text to a structured representation. Entities are retrieved as nodes with textual attributes, and their relationships as edges supplemented with descriptive text. The resulting text-associated graph $G = (V, E, K_\nu, T)$ captures both entities and their semantic relationships, laying the groundwork for later agentic tasks. Here V, E, K_ν, and T denote the set of nodes, edges, node identifiers, and textual chunks (from both nodes and edges), respectively.

3.2 Our Proposed Framework

Our proposed Agentic Graph RAG framework combines contextual reranking with reasoning-based agents to overcome the drawbacks of conventional and graph-based RAG systems. Figure 1 shows the process. The system employs a pipeline with multiple phases:

1. **Query Decomposition.** A reasoning-model-based agent first detects the category of the main query, which may fall into one of four types: *simple* (basic factual questions such as "what," "who," "when," "where"), *sequential* (step-by-step or chronological reasoning), *categorical* (different aspects, perspectives, or types), or *hierarchical* (from general concepts to specific details). Based on the detected category, the query q is decomposed into a set of detailed sub-queries $\{q_1, ..q_n\}$ using prompt-based generation. The generated sub-queries are evaluated against criteria such as sufficiency, length, and uniqueness, with iterative refinement invoked when quality falls short. This self-correcting mechanism improves the reliability of subsequent retrieval and generation phases by ensuring high-quality sub-queries. Figure 2 shows the working principle of this agent.

2. **Keyword Extraction.** From each sub-query, a set of representative keywords K_q is extracted. These serve as anchors for graph-based retrieval, allowing for a more targeted and efficient search over the indexing graph. Decomposing the main query into sub-queries yields more keywords, resulting in more effective retrieval as shown in Table 1.

Table 1: Comparison of Keyword Counts With and Without Query Decomposition Agent

Condition	**Number of Total Extracted Keywords**
Without Query Decomposition Agent	$10 - 15$
With Query Decomposition Agent	$16 - 35$

3. **Relevant Nodes and Paths Retrieval.** Using K_q, relevant nodes $V_q \subseteq V$ are identified from the indexing graph G via semantic embeddings and cosine similarity [11][2]. Multi-hop graph traversal (up to 3 hops) retrieves candidate relational paths in an efficient way, which capture inter-entity connections necessary for complex reasoning and enhancing robustness.

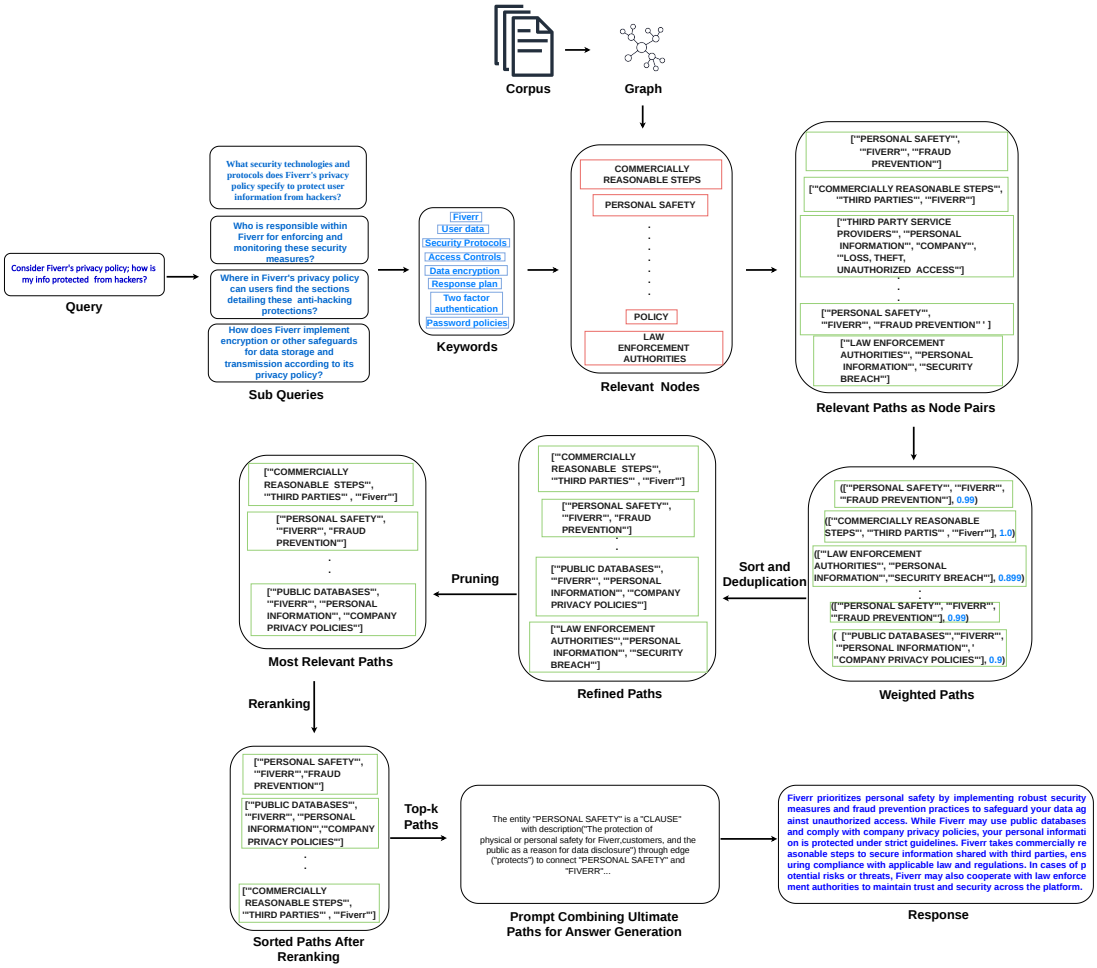


Figure 1: An Overview of the Workflow of Our Framework

4. **Path Weighting.** Inspired by PathRAG [2], each retrieved path $p_i \in P$ is assigned a reliability score $S(p_i)$ through resource allocation across its nodes. The resource originates at the start node and decays as it propagates through neighboring nodes, ensuring that closer nodes contribute more than distant ones. The reliability of a path is then computed as the average resource of its nodes, with higher scores reflecting stronger semantic relevance to the query. After sorting and deduplicating, only the top-N most reliable and highest scored paths are retained for further refinement which enhances efficiency of the pruning agent.

5. **Irrelevant Path Pruning.** Even after weighing and selecting the top-scoring paths, noisy or weakly relevant paths may persist. To address this, a reasoning model integrated within the agent prunes unnecessary candidates based on prompt instructions, as illustrated in Figure 3.

   Formally, the weighted set of candidate paths is denoted as
   $P_q = \{(p_i, S(p_i)) \mid p_i \subseteq P,\ i = 1, \ldots, m\}$

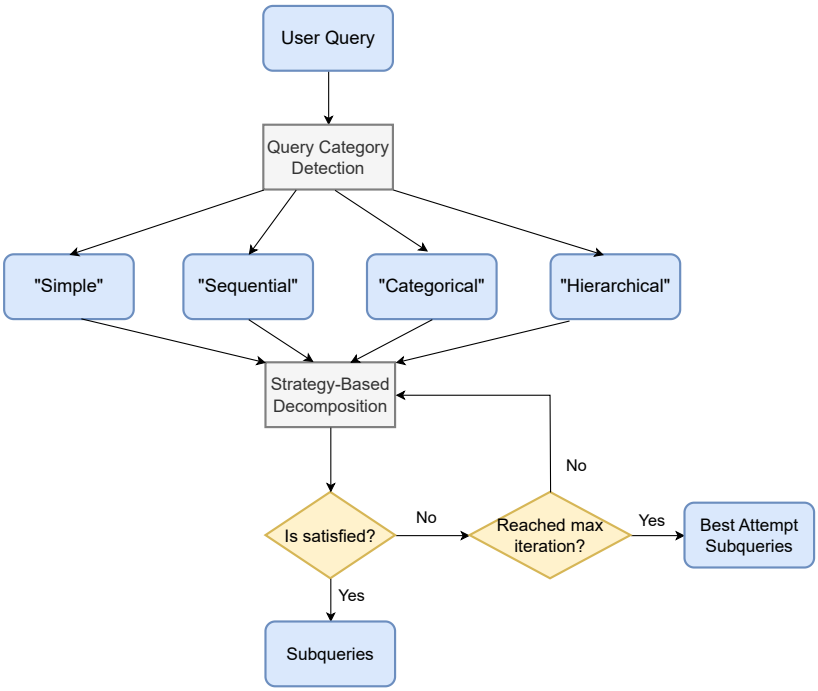


Figure 2: Working Principle of Query Decomposition Agent

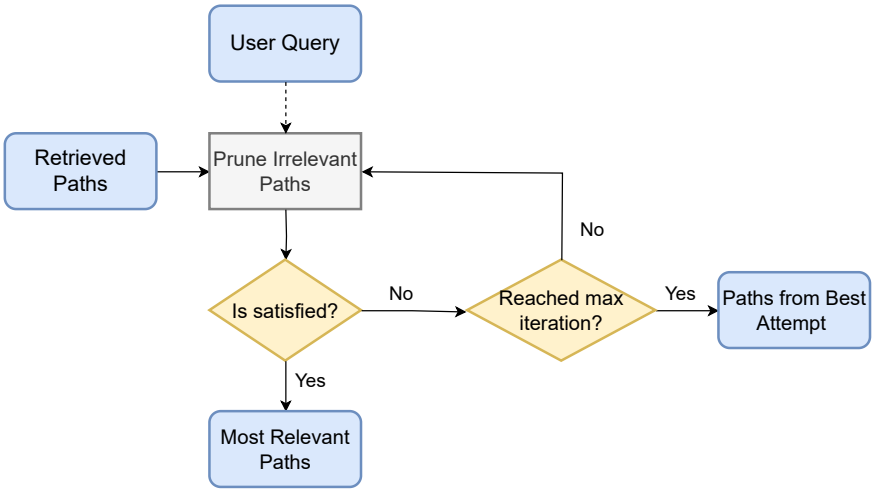


Figure 3: Working Principle of Pruning Agent

where each path p_i has a reliability score $S(p_i)$. The agent iteratively filters out irrelevant or redundant paths, yielding a refined subset:
$P_q^* = \{p_i \in P_q \mid \text{Prune}(q, p_i) = \text{retain}\}$
If P_q^* still contains paths with poor semantic contribution, pruning is performed until a suitable retention rate—defined as the ratio of retained paths to the initial subset—is obtained, or until the maximum number of iterations is met. In the latter case, the agent returns the best subset it encountered. This recurrent filtering ensures that only the most informative and semantically aligned paths are kept for the next phases. In

Table 2: Comparison of Average Number of Paths Before and After Pruning

Condition	**mean_k_before**	**mean_k_after**	**Reduction (%)**
Without Pruning Agent	≈ 360	≈ 360	0%
With Pruning Agent (o4-mini)	≈ 350	≈ 40	88%
With Pruning Agent (Gemini-2.5-Flash)	≈ 350	≈ 190	46%

Table 2, mean_k_before indicates the average number of candidate paths generated before pruning, and mean_k_after represents the remaining paths after the pruning procedure that go for reranking. The pruning agent reduces the path exploration space by up to 88% for o4-mini and 46% for Gemini, effectively filtering noise and improving reasoning efficiency. This also helps to optimize the time complexity of reranking.

6. **Reranking.** To improve retrieval accuracy and efficiency, we employ a reranking phase with neural rerankers to prioritize relevant paths after pruning, as only the top-k reranked paths are used to generate the final prompt for response generation, ensuring that the response is based on the most contextually appropriate evidence while minimizing undue latency.

7. **Response Generation.** The top-k relational paths with their associated textual chunks are incorporated into the final response generation prompt and sent to the LLM to generate response. To avoid the "lost in the middle" effect, the query is placed first, followed by paths sorted in descending reliability, ensuring that the most informative evidence is located in the LLM's golden memory region [13].

This end-to-end system combines decomposition, pruning, and reranking into a structured pipeline to produce contextually rich, trustworthy, and efficient responses. The detailed algorithm is given in Appendix A.

4 Experiments and Results

This section describes the experimental setup and reports results for our framework with and without reranking, along with comparisons to baseline models and additional analysis.

4.1 Experimental Setup

Dataset. We evaluate our framework across three benchmarks, chosen to represent the diverse knowledge-intensive reasoning requirements in E-Business environments.

FinanceBench [8] is a publicly accessible financial QA benchmark, which consists of 150 annotated question-answer pairs grounded in real-world corporate disclosures, earnings reports, and SEC filings. It closely resembles the information requirements of E-Business environments, where precise, multi-step retrieval across vast collections of financial documents is crucial for investment analysis, procurement decisions, and compliance evaluations. We used 90 of the 361 PDF documents in the FinanceBench corpus (roughly 25% of the total) as our retrieval corpus, which covers about 9 million tokens. This scale makes it a realistic and appropriately difficult testbed for assessing graph-based multi-hop retrieval. Furthermore, we made sure that every pair of evaluation questions and answers came from and was firmly rooted in the chosen subset of PDFs. For evaluation, all 150 QA pairs that were accessible to the public were used. The official repository [17] was the source of the dataset.

LegalBench-RAG-mini (LegalBench) [20] is a legal-domain retrieval benchmark focused on high-stakes analytical queries. It is highly relevant to E-Business Information Systems because its legal tasks reflect enterprise activities such as contract analysis, compliance verification, and dispute resolution, which necessitate multi-hop reasoning over complex documents and support explainable retrieval in high-stakes business environments. We worked with a subset of this dataset, using approximately 40% of its document corpus (around 30 documents) and 25% of its QA pairs (195 QAs) for evaluation. The selected corpus comprised approximately 0.8 million tokens, providing a sufficiently large and document-rich retrieval environment for evaluating multi-hop reasoning performance. The dataset was taken from its official Github repository [19].

HotpotQA [29] is a large-scale, Wikipedia-based multi-hop QA benchmark. We sampled approximately 7% of this dataset. The relevant retrieval corpus consisted of the ten Wikipedia paragraphs associated with each sampled query, totaling around 5,000 paragraph instances and 0.75 million tokens after deduplication. We tested our system with 120 QA pairs, focusing on the multi-hop hard subset to ensure a rigorous evaluation. HotpotQA evaluates the framework's capacity to generalize across several domains and knowledge sources. We obtained the dataset from its official Github repository [28].

Due to computational resource constraints, complete evaluation on the full corpora of all three datasets was not feasible. To ensure reproducibility, subsets were generated uniformly at random using a fixed seed. Selected portions were designed to preserve key benchmark characteristics such as query complexity, document length, and reasoning depth.

Models Used for Reasoning and Reranking. In our Agentic Graph RAG framework, Gemini-2.5-Flash (Gemini) and o4-mini are used for reasoning. For reranking, we use ms-marco-MiniLM-L6-v2 (MSM) and bge-reranker-base (BGE), two neural cross-encoder-based rerankers. To evaluate the efficacy of our hybrid approach, experiments are carried out with various combinations of these reasoning and reranking models.

Baselines and Evaluation Measures. We compare our Agentic Graph RAG framework against three graph-based RAG baselines—PathRAG [2], MiniRAG [5], and NodeRAG [27]—to analyze performance-efficiency trade-offs across all three benchmarks. Complementary measurements are used to analyze retrieval and creation performance. RAGAS-based measures such as Context Precision, Context Recall, and Answer Correctness, assess semantic retrieval quality and fidelity. Standard information retrieval (IR) measures [10], such as MRR, Recall@10, and NDCG@10, assess ranking efficacy and document retrieval accuracy. While Context Recall captures semantic coverage of the information needed to answer a query, Recall@10 determines

retrieval of the exact ground-truth document. Context Precision assesses whether the recovered context contains useful information high up in its ranking, hence reducing noise. Meanwhile, MRR and NDCG@10 assess ranking quality, with MRR focusing on how quickly the first relevant item appears and NDCG@10 taking into account the relative position of all relevant papers within the top ten results [25] [10]. ROUGE-L [12] is used to evaluate generation quality, and end-to-end latency is utilized to analyze system efficiency.

Implementation Details. We use gpt-4.1-mini to construct knowledge graphs and generate responses in both our framework and the Graph RAG baselines. The key hyperparameters include a maximum traversal depth of 3 hops, $N = 60$ retrieved nodes, $n = 75\%$ path retention after pruning, and $k = 100$ top-ranked paths for prompt construction. These parameters were chosen based on dataset characteristics and guided by the PathRAG configuration [2]. Additionally, the agents' maximum iterations are set to 2, and the path retention rate threshold for the pruning agent is bounded between 25% and 75%. A 3-hop restriction corresponds to the multi-hop nature of the evaluated benchmarks while avoiding excessive traversal costs, while the other configurations balance candidate coverage, noise reduction, and context efficiency. Experimental environment details are discussed in Appendix B.

4.2 Results and Analysis

The objective of this evaluation was twofold: first, to address **RQ1** by examining how reranking bottlenecks can be optimized while balancing the tradeoffs between efficiency and performance incorporating reasoning models in agents, and second, to address **RQ2** by analyzing how different combinations of reasoning models and rerankers impact multi-hop retrieval quality, decision-making accuracy, and end-to-end efficiency. To this end, we present results in three stages: performance of reasoning model-based agents without rerankers, improvements introduced by incorporating rerankers, and a comparative analysis against existing baseline Graph RAG approaches.

Agents Without Reranker

Table 3 shows the performance of the framework across three datasets when only reasoning model-based agents are employed without reranking. While both o4-mini and Gemini achieve high context precision, their answer correctness remains moderate, and context recall is comparatively lower. Latency is also significant, particularly for Gemini, indicating that reasoning agents alone cannot properly balance accuracy and efficiency, hence producing the baseline bottleneck addressed in **RQ1**.

Table 3: Evaluation Results of Our Agentic Graph RAG Framework (Without Reranking)

Dataset	Reasoning Model	Context Precision	Context Recall	Answer Correctness	ROUGE-L	Overall Latency (s)
FinanceBench	o4-mini	1.00 ± 0.00	0.72 ± 0.04	0.55 ± 0.10	0.19 ± 0.03	116.94 ± 33.50
	Gemini	1.00 ± 0.00	0.68 ± 0.04	0.49 ± 0.05	0.18 ± 0.02	146.08 ± 47.24
LegalBench	o4-mini	0.97 ± 0.06	0.88 ± 0.12	0.56 ± 0.10	0.19 ± 0.03	234.53 ± 72.12
	Gemini	1.00 ± 0.00	0.93 ± 0.05	0.56 ± 0.08	0.18 ± 0.01	337.64 ± 22.63
HotpotQA	o4-mini	0.96 ± 0.20	0.86 ± 0.22	0.70 ± 0.17	0.36 ± 0.12	97.20 ± 26.96
	Gemini	0.98 ± 0.14	0.88 ± 0.23	0.71 ± 0.17	0.37 ± 0.13	153.58 ± 120.17

Agents With Reranker

Table 4 shows a significant improvement in performance following reranker integration. The combination of o4-mini and MSM achieves the highest answer correctness (64% on FinanceBench, 72% on LegalBench, and 74% on HotpotQA) with competitive latency, demonstrating the effectiveness of reranking in improving retrieval accuracy and reasoning consistency. The frequent achievement of 100% context precision implies that agent-guided path pruning and reranking processes effectively filter unnecessary information, guaranteeing highly precise returned contexts. In contrast, Gemini-based combinations show varying accuracy and increased latency on FinanceBench and LegalBench datasets. Notably, ROUGE-L improves with reranking, peaking under the o4-mini and MSM configuration. Overall, o4-mini outperforms Gemini 2.5 Flash in most cases due to its superior instruction-following capabilities and more stable multi-step reasoning in retrieval-augmented settings. Achieving such consistent gains on analytically complex, multi-hop datasets demonstrates the robustness and adaptability of the o4-mini and MSM combination across reasoning settings.

Table 4: Evaluation Results of Our Agentic Graph RAG Framework (With Reranking)

Dataset	**Reasoning Model + Reranker**	**Context Precision**	**Context Recall**	**Answer Correctness**	**ROUGE-L**	**Overall Latency (s)**
FinanceBench	o4-mini+MSM	$\mathbf{1.00 \pm 0.00}$	0.80 ± 0.05	$\mathbf{0.64 \pm 0.05}$	$\mathbf{0.22 \pm 0.11}$	$\mathbf{101.42 \pm 23.36}$
	o4-mini+BGE	0.90 ± 0.03	0.80 ± 0.03	0.49 ± 0.05	0.22 ± 0.11	108.39 ± 32.33
	Gemini+MSM	0.90 ± 0.03	0.73 ± 0.04	0.44 ± 0.10	0.16 ± 0.10	159.80 ± 70.62
	Gemini+BGE	0.95 ± 0.02	$\mathbf{0.85 \pm 0.03}$	0.55 ± 0.02	0.21 ± 0.10	156.95 ± 67.48
LegalBench	o4-mini+MSM	$\mathbf{1.00 \pm 0.00}$	0.97 ± 0.03	$\mathbf{0.72 \pm 0.05}$	$\mathbf{0.20 \pm 0.01}$	$\mathbf{179.67 \pm 9.95}$
	o4-mini+BGE	0.97 ± 0.06	0.90 ± 0.12	0.59 ± 0.06	0.19 ± 0.03	215.95 ± 8.22
	Gemini+MSM	0.97 ± 0.06	0.91 ± 0.03	0.56 ± 0.06	0.18 ± 0.02	270.92 ± 74.17
	Gemini+BGE	1.00 ± 0.00	$\mathbf{0.98 \pm 0.02}$	0.55 ± 0.09	0.19 ± 0.03	304.41 ± 67.73
HotpotQA	o4-mini+MSM	$\mathbf{1.00 \pm 0.00}$	$\mathbf{0.94 \pm 0.13}$	$\mathbf{0.74 \pm 0.14}$	$\mathbf{0.38 \pm 0.10}$	$\mathbf{97.08 \pm 32.86}$
	o4-mini+BGE	1.00 ± 0.00	0.94 ± 0.16	0.72 ± 0.15	0.37 ± 0.13	114.12 ± 54.95
	Gemini+MSM	1.00 ± 0.00	0.92 ± 0.18	0.72 ± 0.16	0.38 ± 0.12	105.40 ± 84.4
	Gemini+BGE	1.00 ± 0.00	0.92 ± 0.18	0.65 ± 0.19	0.38 ± 0.12	102.60 ± 37.50

Table 5 illustrates how rerankers improve context prioritization and retrieval relevance. The o4-mini and MSM combination achieves the best overall retrieval performance, obtaining the highest Recall@10 across all datasets and perfect MRR on HotpotQA, demonstrating its ability to consistently retrieve relevant contexts. While the o4-mini and BGE combination achieves slightly higher NDCG@10 on HotpotQA, Gemini-based setups generally underperform in Recall@10 and NDCG@10 across datasets, despite comparable MRR values in some cases. Figure 4 further visualizes comparative retrieval performance using Recall@10 and NDCG@10. Overall, these results indicate that reranking significantly enhances retrieval relevance and ranking quality, reducing retrieval bottlenecks and improving reasoning-driven retrieval performance, addressing **RQ2**.

Figure 5 shows the overall latency comparison across model combinations for all three datasets. MSM consistently outperforms BGE in latency when combined with o4-mini. The pattern is less consistent with Gemini: MSM gains a significant advantage on LegalBench (270.92s vs. 304.41s), while BGE is slightly faster on FinanceBench (156.95s vs. 159.80s) and HotpotQA (102.60s vs. 105.40s), implying that MSM's latency advantage is most noticeable when

Table 5: Retrieval and Ranking Performance of Our Agentic Graph RAG Framework

Dataset	**Reasoning Model + Reranker**	**Recall@10**	**NDCG@10**	**MRR**
FinanceBench	o4-mini+MSM	$\mathbf{0.59 \pm 0.02}$	$\mathbf{0.74 \pm 0.02}$	$\mathbf{0.98 \pm 0.01}$
	o4-mini+BGE	0.35 ± 0.03	0.71 ± 0.02	0.97 ± 0.02
	Gemini+MSM	0.16 ± 0.01	0.70 ± 0.02	0.93 ± 0.03
	Gemini+BGE	0.17 ± 0.02	0.68 ± 0.02	0.96 ± 0.02
LegalBench	o4-mini+MSM	$\mathbf{0.20 \pm 0.11}$	$\mathbf{0.67 \pm 0.05}$	0.96 ± 0.04
	o4-mini+BGE	0.13 ± 0.01	0.64 ± 0.08	0.92 ± 0.09
	Gemini+MSM	0.08 ± 0.01	0.59 ± 0.01	$\mathbf{0.97 \pm 0.06}$
	Gemini+BGE	0.09 ± 0.06	0.58 ± 0.04	0.94 ± 0.06
HotpotQA	o4-mini+MSM	$\mathbf{0.47 \pm 0.27}$	0.83 ± 0.10	$\mathbf{1.00 \pm 0.00}$
	o4-mini+BGE	0.45 ± 0.27	$\mathbf{0.86 \pm 0.11}$	0.98 ± 0.12
	Gemini+MSM	0.26 ± 0.20	0.71 ± 0.14	0.98 ± 0.12
	Gemini+BGE	0.28 ± 0.23	0.75 ± 0.12	0.97 ± 0.13

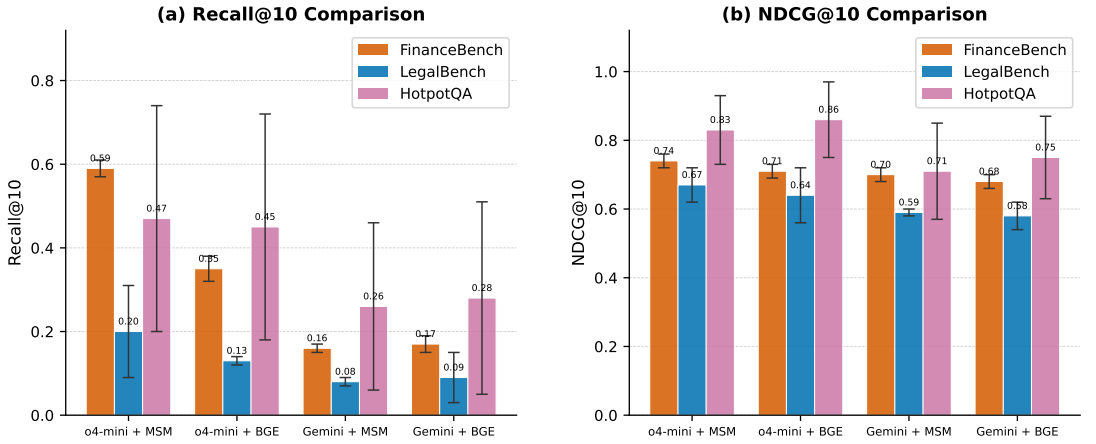


Figure 4: Comparative Retrieval Performance: Recall@10 and NDCG@10 Across Model Combinations in Our Framework

paired with the lighter o4-mini reasoning model. Latency variations also reflect dataset complexity. LegalBench and FinanceBench involve long legal documents and dense financial data, respectively, demanding more complex reasoning and increasing computational cost. These results highlight the correlation between computational cost and corpus complexity — an important consideration in E-Business applications involving financial and legal documents. Overall, the o4-mini and MSM combination reduces the reranking bottleneck and addresses the efficiency–performance trade-offs posed in **RQ1**, achieving an optimal balance between retrieval performance and efficiency.

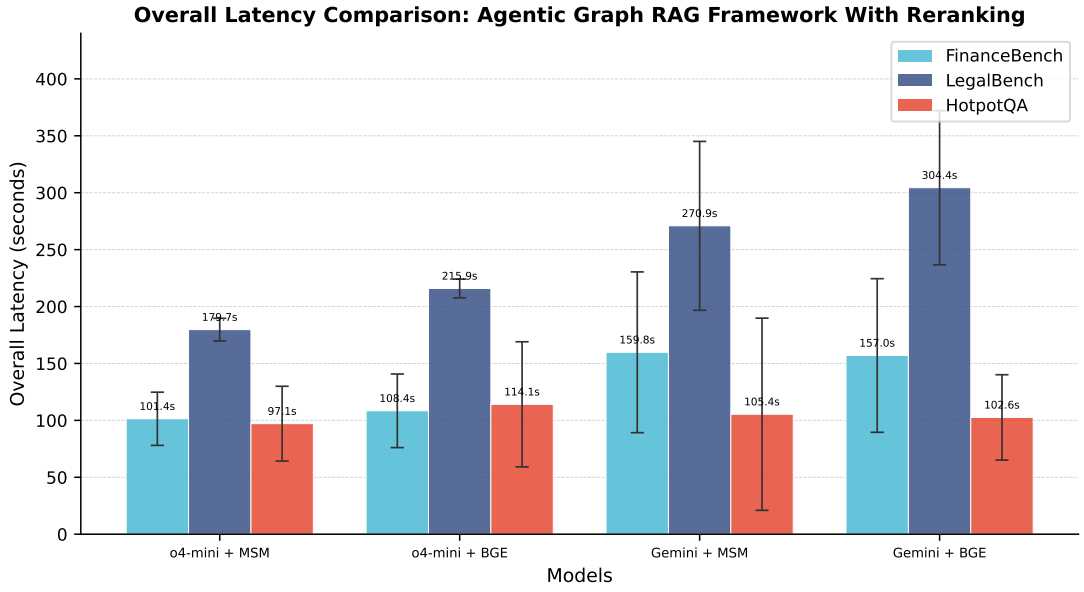


Figure 5: Overall Latency Comparison of Our Framework Combining Different Combinations of Models

Our Framework vs Baseline Models

Table 6 and Figure 6 compare our best-performing configuration (o4-mini+MSM) against PathRAG, MiniRAG, and NodeRAG across all three datasets. Our framework clearly demonstrates its advantage in the knowledge-intensive, multi-hop retrieval scenarios essential to E-Business environments by consistently achieving the highest Answer Correctness and Context Recall across all datasets, with the most noticeable gains on FinanceBench and LegalBench. While all three baselines have lower latency, this speed comes at a measurable cost in retrieval completeness and answer quality, as demonstrated consistently across Context Recall, Answer Correctness, and ROUGE-L. NodeRAG achieves exceptionally low latency by substituting deep path traversal with a shallow Personalized PageRank algorithm, hence avoiding the agent reasoning and reranking stages that our system uses to obtain improved accuracy. The accuracy benefits of our methodology outweigh the extra computing expense in use cases where accuracy and evidence traceability are crucial, such as financial analysis, legal compliance, and organizational decision-making.

Our results demonstrate that the proposed Agentic Graph RAG framework effectively answers both research questions. By removing unnecessary paths early, combining reasoning model-based agents with contextual reranking for **RQ1** mitigates the reranking bottleneck, yielding a comparatively tolerable computational cost without sacrificing retrieval accuracy. For **RQ2**, the combination of efficient reranking (MSM) and lightweight reasoning (o4-mini) consistently provides the optimum balance of practical efficiency, answer correctness, and retrieval relevance across all three benchmarks. While standard deviations of some baselines overlap due to variations in query complexity, the method shows consistent average improvements across datasets and settings, suggesting stable empirical gains rather than dataset-specific effects. As the FinanceBench results clearly show, these findings are particularly pertinent to E-Business

Table 6: Evaluation Results: Comparison Between Our Framework and Baseline Models

Dataset	Reasoning Model + Reranker	Context Precision	Context Recall	Answer Correctness	ROUGE-L	Overall Latency (s)
FinanceBench	Our Framework (o4-mini + MSM)	**1.00 ± 0.00**	**0.80 ± 0.05**	**0.64 ± 0.05**	**0.22 ± 0.11**	101.42 ± 23.36
	PathRAG	0.90 ± 0.03	0.73 ± 0.04	0.47 ± 0.02	0.13 ± 0.05	24.18 ± 9.96
	MiniRAG	0.45 ± 0.09	0.56 ± 0.03	0.28 ± 0.09	0.15 ± 0.02	20.00 ± 3.33
	NodeRAG	0.97 ± 0.03	0.79 ± 0.03	0.55 ± 0.01	0.21 ± 0.02	**17.00 ± 6.67**
LegalBench	Our Framework (o4-mini + MSM)	**1.00 ± 0.00**	**0.97 ± 0.03**	**0.72 ± 0.05**	0.20 ± 0.01	179.67 ± 9.95
	PathRAG	1.00 ± 0.00	0.89 ± 0.07	0.63 ± 0.04	0.20 ± 0.01	146.22 ± 55.63
	MiniRAG	1.00 ± 0.00	0.96 ± 0.01	0.64 ± 0.10	**0.23 ± 0.08**	183.79 ± 30.85
	NodeRAG	0.97 ± 0.06	0.89 ± 0.05	0.61 ± 0.02	0.20 ± 0.01	**6.18 ± 1.75**
HotpotQA	Our Framework (o4-mini + MSM)	**1.00 ± 0.00**	**0.94 ± 0.13**	**0.74 ± 0.14**	**0.38 ± 0.10**	97.08 ± 32.86
	PathRAG	1.00 ± 0.00	0.86 ± 0.23	0.71 ± 0.17	0.38 ± 0.11	8.70 ± 1.27
	MiniRAG	0.97 ± 0.14	0.90 ± 0.19	0.73 ± 0.17	0.37 ± 0.12	7.70 ± 2.30
	NodeRAG	0.97 ± 0.13	0.84 ± 0.21	0.62 ± 0.15	0.28 ± 0.03	**6.47 ± 3.24**

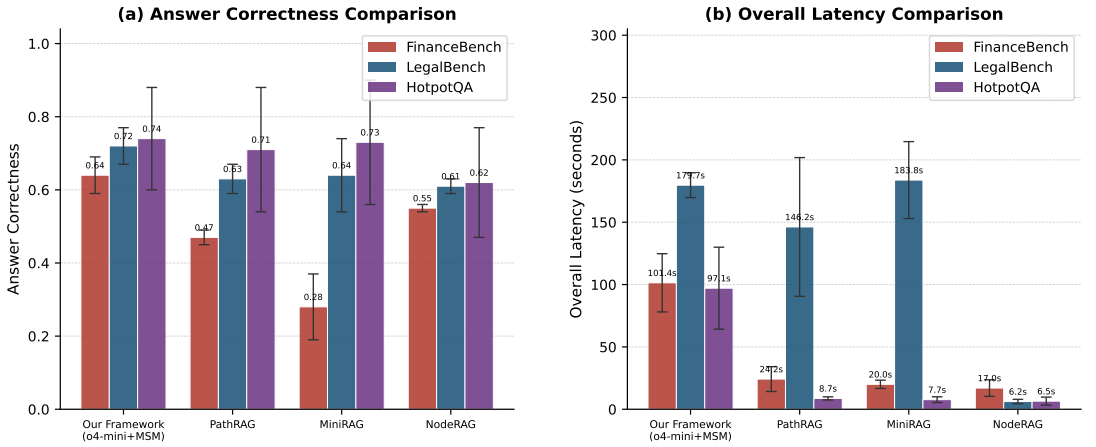


Figure 6: Evaluation Results: Comparison Between Our Framework and Baseline Models

Information Systems, where precise, traceable, and practically effective access to complicated financial knowledge enables well-informed operational and strategic decision-making.

5 Conclusion

This research presents a reasoning-driven Agentic Graph RAG framework that combines lightweight reasoning agents with contextual reranking to improve retrieval precision and efficiency in multi-hop reasoning for E-Business Information Systems. Combining reasoning models

like o4-mini and Gemini-2.5-Flash within agents, as well as integrating rerankers like ms-marco-MiniLM-L6-v2 and bge-reranker-base, improves both reasoning quality and response relevance in our system. Three benchmarks covering different knowledge domains are used to assess the framework: FinanceBench, which covers real-world business disclosures and financial data that directly reflect E-Business information demands; LegalBench-RAG-mini, which represents high-stakes analytical reasoning; and HotpotQA, which evaluates generalizability across broader knowledge domains. The o4-mini and MSM combination achieved the optimal balance of accuracy and latency, while Gemini-based configurations incurred higher computational costs without proportional gains in retrieval accuracy or answer correctness. These findings validate the framework's adaptability across multiple knowledge domains and its suitability for E-Business environments where accurate, traceable, and trustworthy multi-hop knowledge access is essential for informed decision-making.

6 Future Work

There are several directions for further research. First, the methodology should be tested on larger, more diverse corpora to ensure its generalizability, alongside exploring architectural optimizations to further reduce end-to-end latency. Second, a thorough sensitivity study of critical hyperparameters such as traversal depth, retrieved nodes, path retention rate, and reranking size is required to better understand their impact on retrieval quality and efficiency. Third, support for dynamic knowledge graph updates would increase applicability in changing real-world E-Business environments. Finally, future research will look into multimodal Agentic Graph RAG and adaptive retrieval-reranking techniques to improve reasoning, scalability, and flexibility across a variety of information settings.

References

[1] Chia-Yuan Chang, Zhimeng Jiang, Vineeth Rakesh, Menghai Pan, Chin-Chia Michael Yeh, Guanchu Wang, Mingzhi Hu, Zhichao Xu, Yan Zheng, Mahashweta Das, et al. Main-rag: Multi-agent filtering retrieval-augmented generation. *ArXiv*, abs/2501.00332, 2024. Available at https://api.semanticscholar.org/CorpusID:275212408.

[2] Boyu Chen, Zirui Guo, Zidan Yang, Yuluo Chen, Junze Chen, Zhenghao Liu, Chuan Shi, and Cheng Yang. Pathrag: Pruning graph-based retrieval augmented generation with relational paths. *ArXiv*, abs/2502.14902, 2025. Available at https://api.semanticscholar.org/CorpusID:276558341.

[3] Chanyeol Choi, Jihoon Kwon, Minjae Kim, et al. FinAgentBench: A benchmark dataset for agentic retrieval in financial question answering. *arXiv preprint arXiv:2508.14052*, 2025.

[4] Jialin Dong, Bahare Fatemi, Bryan Perozzi, Lin F. Yang, and Anton Tsitsulin. Don't forget to connect! improving rag with graph-based reranking. *ArXiv*, abs/2405.18414, 2024. Available at https://api.semanticscholar.org/CorpusID:270067901.

[5] Tianyu Fan, Jingyuan Wang, Xubin Ren, and Chao Huang. Minirag: Towards extremely simple retrieval-augmented generation. *ArXiv*, abs/2501.06713, 2025. Available at https://api.semanticscholar.org/CorpusID:275470564.

[6] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jin Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Qianyu Guo, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *ArXiv*, abs/2312.10997, 2023. Available at https://api.semanticscholar.org/CorpusID:266359151.

[7] Michael Glass, Gaetano Rossiello, Md Faisal Mahbub Chowdhury, Ankita Naik, Pengshan Cai, and Alfio Gliozzo. Re2G: Retrieve, rerank, generate. In Marine Carpuat, Marie-Catherine de Marneffe,

and Ivan Vladimir Meza Ruiz, editors, *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2701–2715, Seattle, United States, July 2022. Association for Computational Linguistics. Available at "https://aclanthology.org/2022.naacl-main.194/".

[8] Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. FinanceBench: A new benchmark for financial question answering. *arXiv preprint arXiv:2311.11944*, 2023.

[9] Jisoo Jang and Wen-Syan Li. Au-rag: Agent-based universal retrieval augmented generation. In *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*, SIGIR-AP 2024, page 2–11, New York, NY, USA, 2024. Association for Computing Machinery. Available at https://doi.org/10.1145/3673791.3698416.

[10] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of ir techniques. *ACM Trans. Inf. Syst.*, 20(4):422–446, October 2002. Available at https://doi.org/10.1145/582415.582418.

[11] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu, editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online, November 2020. Association for Computational Linguistics. Available at "https://aclanthology.org/2020.emnlp-main.550/".

[12] Chin-Yew Lin. Rouge: A package for automatic evaluation of summaries. In *Proceedings of the ACL Workshop: Text Summarization Branches Out 2004*, page 10, 01 2004.

[13] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2023. Available at https://api.semanticscholar.org/CorpusID:259360665.

[14] Haoran Luo, E. Haihong, Guanting Chen, Qika Lin, Yikai Guo, Fangzhi Xu, Zemin Kuang, Meina Song, Xiaobao Wu, Yifan Zhu, and Anh Tuan Luu. Graph-r1: Towards agentic graphrag framework via end-to-end reinforcement learning. *ArXiv*, abs/2507.21892, 2025. Available at https://api.semanticscholar.org/CorpusID:280337494.

[15] Thang Nguyen, Peter Chin, and Yu-Wing Tai. Ma-rag: Multi-agent retrieval-augmented generation via collaborative chain-of-thought reasoning. *ArXiv*, abs/2505.20096, 2025. Available at https://api.semanticscholar.org/CorpusID:278911373.

[16] Tong Niu, Shafiq Joty, Ye Liu, Caiming Xiong, Yingbo Zhou, and Semih Yavuz. Judgerank: Leveraging large language models for reasoning-intensive reranking. *ArXiv*, abs/2411.00142, 2024. Available at https://api.semanticscholar.org/CorpusID:273798418.

[17] Patronus AI. FinanceBench dataset repository. https://github.com/patronus-ai/financebench, 2023.

[18] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. Graph retrieval-augmented generation: A survey. *ACM Trans. Inf. Syst.*, 44(2), December 2025. Available at https://doi.org/10.1145/3777378.

[19] Nicholas Pipitone and Ghita Houir Alami. Legalbench-rag. GitHub Repository, 2024. Available at https://github.com/zeroentropy-ai/legalbenchrag.

[20] Nicholas Pipitone and Ghita Houir Alami. Legalbench-rag: A benchmark for retrieval-augmented generation in the legal domain. *ArXiv*, abs/2408.10343, 2024. Available at https://api.semanticscholar.org/CorpusID:271909426.

[21] Aditi Singh, Abul Ehtesham, Saket Kumar, and Tala Talaei Khoei. Agentic retrieval-augmented generation: A survey on agentic rag. *ArXiv*, abs/2501.09136, 2025. Available at https://api.semanticscholar.org/CorpusID:275570331.

[22] Kuldeep Singh, Simerjot Kaur, and Charese Smiley. FinQAPT: Empowering financial decisions with end-to-end LLM-driven question answering pipeline. In *5th ACM International Conference on AI in Finance (ICAIF '24)*, 2024.

[23] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Sai Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung yeung Shum, and Jian Guo. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In *International Conference on Learning Representations*, 2023. Available at https://api.semanticscholar.org/CorpusID:263333907.

[24] Yixuan Tang and Yi Yang. Multihop-rag: Benchmarking retrieval-augmented generation for multi-hop queries. *ArXiv*, abs/2401.15391, 2024. Available at https://api.semanticscholar.org/CorpusID:267312593.

[25] Ellen M. Voorhees. The trec-8 question answering track report. In *Proceedings of the 8th Text Retrieval Conference (TREC)*, volume 99, pages 77–82, 1999.

[26] Zhishang Xiang, Chuanjie Wu, Qinggang Zhang, Shengyuan Chen, Zijin Hong, Xiao Huang, and Jinsong Su. When to use graphs in rag: A comprehensive analysis for graph retrieval-augmented generation. *ArXiv*, abs/2506.05690, 2025. Available at https://api.semanticscholar.org/CorpusID:279244678.

[27] Tianyang Xu, Haojie Zheng, Chengze Li, Haoxiang Chen, Yixin Liu, Ruoxi Chen, and Lichao Sun. Noderag: Structuring graph-based rag with heterogeneous nodes. *ArXiv*, abs/2504.11544, 2025. Available at https://api.semanticscholar.org/CorpusID:277824631.

[28] Zhilin Yang and Peng Qi. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. GitHub Repository, 2018. Dataset Available at https://github.com/hotpotqa/hotpot.

[29] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii, editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium, October-November 2018. Association for Computational Linguistics. Available at "https://aclanthology.org/D18-1259/".

[30] Yue Yu, Wei Ping, Zihan Liu, Boxin Wang, Jiaxuan You, Chao Zhang, Mohammad Shoeybi, and Bryan Catanzaro. Rankrag: unifying context ranking with retrieval-augmented generation in llms. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA, 2024. Curran Associates Inc.

[31] Le Zhang, Bo Wang, Xipeng Qiu, Siva Reddy, and Aishwarya Agrawal. Rearank: Reasoning re-ranking agent via reinforcement learning. *ArXiv*, abs/2505.20046, 2025. Available at https://api.semanticscholar.org/CorpusID:278910905.

[32] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Z. Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jianyun Nie, and Ji rong Wen. A survey of large language models. *ArXiv*, abs/2303.18223, 2023. Available at https://api.semanticscholar.org/CorpusID:257900969.

[33] Xinjie Zhao, Moritz Blum, Rui Yang, Boming Yang, Luis Marquez-Carpintero, Monica Pina-Navarro, Tony Wang, Xin Li, Huitao Li, Yanran Fu, Rongrong Wang, Juntao Zhang, and Irene Li. Agentigraph: An interactive knowledge graph platform for llm-based chatbots utilizing private data. *ArXiv*, abs/2410.11531, 2024. Available at https://api.semanticscholar.org/CorpusID:273351391.

[34] Xiangrong Zhu, Yuexiang Xie, Yi Liu, Yaliang Li, and Wei Hu. Knowledge graph-guided retrieval augmented generation. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8912–8924, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. Available at "https://aclanthology.org/2025.naacl-long.449/".

Appendix

A Algorithm

This section describes the detailed algorithmic approach for our proposed Agentic Graph RAG system, which operates in 7 steps as illustrated in Algorithm 1.

Algorithm 1: `AgenticGraphRAG`(q, G, V_{ent}, V_{rel})

input : q, G, V_{ent}, V_{rel} ▷ query, knowledge graph, entity vectorDB, relationship vectorDB
output: R ▷ generated response

$SubQueries[\,] \leftarrow$ `DecomposeQuery`(q) ▷ Step 1: Query Decomposition
$Keywords[\,] \leftarrow$ `init()`;
for $sub \leftarrow SubQueries[0]$ **to** $SubQueries[length-1]$ **do**
  $kw \leftarrow$ `ExtractKeywords`(sub) ▷ Step 2: Keyword Extraction
  ; $Keywords$.`add`(kw);
$EntityContext \leftarrow V_{ent}$.`Query`($Keywords$) ▷ Step 3: Context Retrieval
$RelationContext \leftarrow V_{rel}$.`Query`($Keywords$);
$SourceNodes[\,] \leftarrow$ `ExtractNodes`($EntityContext$);
$AllPaths[\,] \leftarrow$ `init()`
for $i \leftarrow 0$ **to** $length(SourceNodes)-1$ **do**
  for $j \leftarrow i+1$ **to** $length(SourceNodes)-1$ **do**
    $paths \leftarrow$ `FindPaths`(G, $SourceNodes[i]$, $SourceNodes[j]$, $maxDepth = 3$);
    $AllPaths$.`addAll`($paths$);
$ScoredPaths[\,] \leftarrow$ `init()` ▷ Step 4: Path Scoring
for $path \leftarrow AllPaths[0]$ **to** $AllPaths[length-1]$ **do**
  $score \leftarrow$ `CalculateWeight`($path$);
  $ScoredPaths$.`add`($\langle path, score \rangle$);
`sort`($ScoredPaths$ by $score$ descending);
$topN \leftarrow n\% \times length(ScoredPaths)$;
$TopPaths \leftarrow ScoredPaths[0 : topN]$;
$CandidatePaths[\,] \leftarrow$ `ExtractPaths`($TopPaths$) ▷ Step 5: Path Pruning
$FilteredPaths \leftarrow$ `PruneIrrelevantPaths`(q, $CandidatePaths$);
$RerankInputs[\,] \leftarrow$ `init()` ▷ Step 6: Neural Reranking
for $path \leftarrow FilteredPaths[0]$ **to** $FilteredPaths[length-1]$ **do**
  $input \leftarrow q +$ "[SEP]" + `PathToString`($path$);
  $RerankInputs$.`add`($input$);
$Scores[\,] \leftarrow$ `RerankerModel`($RerankInputs$);
$RerankedPaths \leftarrow$ `SortByScore`($FilteredPaths$, $Scores$);
$FinalPaths \leftarrow RerankedPaths[0 : k]$;
$Context \leftarrow$ `BuildNarrativeContext`($FinalPaths$, G)
$SystemPrompt \leftarrow$ `FormatContext`($Context$) ▷ Step 7: Response Generation
$R \leftarrow$ `LLM`(q, $SystemPrompt$);
return R;

B Computational Cost & Resource Requirements

In this section, we demonstrate the experimental environment and analyze the cost required for our experiments.

B.1 Experimental Environment

All experiments were conducted using a single NVIDIA T4 GPU (16 GB VRAM). Each instance was equipped with 16-32 vCPUs and 64 GB of system memory, giving sufficient resources for concurrent reasoning, reranking, and retrieval activities.

B.2 Cost Analysis

To analyze the proposed framework's operational efficiency, we estimated the computing costs incurred in three important stages: graph generation, reasoning, and response generation, which are explained below.

B.2.1 Graph Generation.

To create structured graphs of entity–relation mappings, approximately 10.6 million source tokens from all three corpora were combined during the ingestion phase. Because this method maps both dense entity–relation triplets and extensive text-associated node and edge properties, constructing the index graph required an output of approximately 20 million tokens. This substantial indexing phase was carried out using GPT-4.1-mini, resulting in an estimated total graph generation cost of USD 36.24.

B.2.2 Reasoning.

Our agents' reasoning model processed between 500 and 1,200 tokens per query and produced about 200 to 800 tokens in response during the reasoning phase, which includes query decomposition and pruning of irrelevant paths. This resulted in a remarkably low cost of about USD 0.006 per query.

B.2.3 Response Generation.

Every query that was sent to GPT-4.1-mini generated an average of 500 output tokens and contained approximately 15,000–16,000 input tokens, which included the context and query for response generation. The resulting per-query cost was approximately USD 0.007.

Overall, these findings show that even while the suggested framework incorporates several reasoning models into the agents, it still maintains computational efficiency.

C Error Analysis in Multi-Hop Reasoning

Our Agentic Graph RAG framework has robust error-handling features to maintain stability during multi-hop reasoning. Potential mistakes in the Query Decomposition phase, such as malformed outputs, or incorrect sub-query counts, are automatically discovered and corrected. When such failures occur, the decomposition agent returns to the original core query, allowing the reasoning process to proceed. Furthermore, ambiguity, over-decomposition, or too short

sub-queries result in iterative refining until a legitimate, semantically coherent set of sub-queries is obtained.

In the Pruning Irrelevant Paths phase, failures may arise from LLM parsing issues, semantic misalignment, or incomplete filtering of irrelevant paths. When reasoning model failures, timeouts, or low retention rates are discovered, the agent implements multi-level fallback solutions such as a keyword-based pruning heuristic or a full-path return mechanism. This ensures that the retrieval process continues uninterrupted and that the reranking stage receives consistent inputs.

Empirical analysis of the datasets revealed that these recovery mechanisms successfully maintained pipeline continuity despite model or server failures. Such errors occur very rarely; however, when they do arise, they are mostly caused by ambiguous path construction rather than pruning or LLM parsing issues, occasionally resulting in slightly lower recall and increased latency. Overall correctness was maintained since the framework efficiently addressed these problems. These findings support the framework's resilience and reliability in managing complex reasoning operations and dynamic query contexts.